

# **Snowflake RBAC & Access Management**

## **1. Introduction to Access Management in Snowflake**

Snowflake uses **Role-Based Access Control (RBAC)** to manage who can access what. Instead of giving permissions directly to users, Snowflake grants **privileges to roles**, and then **roles are assigned to users**. This makes security scalable and manageable in real-world projects.

Key idea:

**Privileges → Roles → Users**

Users activate a role to inherit its privileges.

---

## **2. System-Defined Roles in Snowflake**

Snowflake provides several built-in roles that exist in every account:

### **2.1 ACCOUNTADMIN**

- Highest-level role
- Owner of the Snowflake account
- Full access to all objects, billing, resource monitors, security, and data
- Typically held by the DBA or platform admin team
- Not recommended for day-to-day development or production usage

**Example scenario:**

ACCOUNTADMIN can see cost usage and create resource monitors.

**Example query:**

```
SHOW RESOURCE MONITORS;
```

**Explanation:**

Only ACCOUNTADMIN has the privilege to view and manage account-level cost controls like resource monitors.

---

### **2.2 SYSADMIN**

- Used for **creating and managing databases and virtual warehouses**
- Owns most data objects in real projects
- All custom roles are usually placed under SYSADMIN

**Example query:**

```
CREATE DATABASE PROD_DB;
```

**Explanation:**

SYSADMIN is responsible for data objects. Creating databases under SYSADMIN ensures proper ownership and inheritance.

---

**2.3 SECURITYADMIN**

- Manages **grants and revokes**
- Can create roles and assign privileges
- Does NOT create data objects

**Example query:**

```
GRANT SELECT ON TABLE PROD_DB.PUBLIC.SALES TO ROLE ANALYST_ROLE;
```

**Explanation:**

SECURITYADMIN is responsible for controlling access, not data creation.

---

**2.4 USERADMIN**

- Creates and manages users
- Assigns default roles
- Cannot grant data privileges

**Example query:**

```
CREATE USER navin PASSWORD='Temp@123';
```

**Explanation:**

USERADMIN manages users but does not control what data they can access.

---

**2.5 PUBLIC**

- Default role assigned to every user
- Minimal privileges
- Should not be used for granting business access

**Example query:**

```
USE ROLE PUBLIC;
```

**Explanation:**

PUBLIC has almost no privileges and is mainly a placeholder role.

---

### 3. Role Hierarchy in Snowflake

Roles in Snowflake form a **hierarchy**. When one role is granted to another role, the receiving role **inherits all privileges**.

Standard hierarchy:

```
ACCOUNTADMIN
├─ SYSADMIN
├─ Custom Roles (ETL_ROLE, ANALYST_ROLE, etc.)
```

This ensures ACCOUNTADMIN always has visibility and control over all objects.

#### Example query:

```
GRANT ROLE SNOWFLAKE_DEMO_ROLE TO ROLE SYSADMIN;
```

#### Explanation:

By placing the custom role under SYSADMIN, both SYSADMIN and ACCOUNTADMIN inherit its privileges.

---

### 4. Privileges in Snowflake

Privileges define **what action is allowed** on an object.

Common privileges:

- USAGE (database, schema, warehouse)
- SELECT, INSERT, UPDATE, DELETE (tables)
- CREATE (schemas, tables)

Privileges are always granted to **roles**, never directly to users.

#### Example query:

```
GRANT USAGE ON DATABASE PROD_DB TO ROLE SNOWFLAKE_DEMO_ROLE;
```

#### Explanation:

Without USAGE, a role cannot even see the database, regardless of table permissions.

---

### 5. Creating Users in Snowflake

Users represent individuals or service accounts.

Steps:

1. Create user
2. Assign default role
3. Explicitly grant the role

**Example query:**

```
CREATE USER navin  
PASSWORD='Welcome@123'  
DEFAULT_ROLE=SYSADMIN  
MUST_CHANGE_PASSWORD=TRUE;
```

**Explanation:**

This creates the user, but privileges are not active until the role is granted.

**Grant role:**

```
GRANT ROLE SYSADMIN TO USER navin;
```

---

## 6. Creating Custom Roles

Custom roles represent **job functions** (ETL, Reporting, Read-only, etc.).

**Example query:**

```
CREATE ROLE SNOWFLAKE_DEMO_ROLE;
```

**Explanation:**

Instead of granting access to users individually, access is granted to roles.

---

## 7. Granting Access to Databases and Schemas

Access must be granted step-by-step:

1. USAGE on database
2. USAGE on schema
3. Privileges on tables

**Example queries:**

```
GRANT USAGE ON DATABASE PROD_DB TO ROLE SNOWFLAKE_DEMO_ROLE;  
GRANT USAGE ON SCHEMA PROD_DB.PUBLIC TO ROLE SNOWFLAKE_DEMO_ROLE;
```

**Explanation:**

USAGE is mandatory to access lower-level objects.

---

## 8. Granting Table-Level Access

To allow read access:

### Example query:

```
GRANT SELECT ON ALL TABLES IN SCHEMA PROD_DB.PUBLIC TO ROLE SNOWFLAKE_DEMO_ROLE;
```

### Explanation:

This allows SELECT access on existing tables only.

---

## 9. Future Grants (Very Important)

Snowflake does NOT automatically grant access to newly created tables.

To ensure continued access, **future grants** are required.

### Example query:

```
GRANT SELECT ON FUTURE TABLES IN SCHEMA PROD_DB.PUBLIC TO ROLE SNOWFLAKE_DEMO_ROLE;
```

### Explanation:

Any new table created later in this schema will automatically be accessible.

---

## 10. Assigning Roles to Users

Once roles are ready, assign them to users.

### Example query:

```
GRANT ROLE SNOWFLAKE_DEMO_ROLE TO USER navin;
```

### Explanation:

User can switch roles and access data based on assigned roles.

---

## 11. Switching Roles

Users can switch roles globally or per worksheet.

### Example query:

```
USE ROLE SNOWFLAKE_DEMO_ROLE;
```

### Explanation:

Privileges available depend on the active role.

---

## 12. Revoking Privileges

Privileges can be removed using REVOKE.

### Example query:

```
REVOKE SELECT ON ALL TABLES IN SCHEMA PROD_DB.PUBLIC FROM ROLE  
SNOWFLAKE_DEMO_ROLE;
```

### Explanation:

Immediately removes access from all users holding the role.

---

## 13. Real-Time Access Management (Industry Practice)

- Access is granted at **team-role level**, not individual users
- Examples: ETL\_ROLE, ANALYST\_ROLE, READ\_ONLY\_ROLE
- Service accounts are used for tools

### Example:

All ETL developers use ETL\_ROLE. Granting or revoking affects the entire team.

---

## 14. Authentication Methods

### 14.1 Username & Password

- Basic authentication
- Often combined with MFA

### 14.2 Multi-Factor Authentication (MFA)

- OTP or push notification
- Enhances login security

### 14.3 Key Pair Authentication (Recommended)

- Used by ETL tools and Python
- More secure than passwords

### Example command (conceptual):

```
ALTER USER etl_user SET RSA_PUBLIC_KEY='MIIBIjANBgkq...';
```

### Explanation:

Private key stays with the application, public key is stored in Snowflake.

---

## 15. Snowpipe vs CI/CD

### Snowpipe

- Used for near real-time data ingestion
- Event-driven

### CI/CD (GitHub Actions / Bitbucket)

- Used for deployments
- Manages SQL scripts, schema changes

#### Key difference:

Snowpipe loads data, CI/CD deploys code.

---

## 16. Key Takeaways

- Privileges are never granted directly to users
  - Roles are the core of Snowflake security
  - Always use future grants
  - Custom roles should be placed under SYSADMIN
  - ACCOUNTADMIN should be used sparingly
- 

These notes are aligned with real-time Snowflake project practices and interview expectations.

## **Snowflake End-to-End RBAC Scenario (Real Project Style)**

This is a **complete, realistic RBAC implementation** exactly how it is done in enterprise Snowflake projects.

---

### **Project Scenario**

**Company:** ABC Retail Ltd

**Use case:** Sales analytics platform

#### **Teams involved:**

- **DBA Team** – Manages Snowflake account
- **ETL Team** – Loads data into Snowflake
- **Analytics Team** – Reads data for reports

#### **Requirements:**

- ETL team can **insert/update/delete** data
  - Analytics team can **only read** data
  - No direct access to users
  - Future tables should automatically inherit access
  - ACCOUNTADMIN should have full visibility
- 

### **Step 1: Role Design (Very Important)**

#### **Roles needed:**

- ETL\_ROLE
- ANALYST\_ROLE

Custom roles are always created under **SYSADMIN**.

---

### **Step 2: Create Roles (SECURITYADMIN)**

```
CREATE ROLE ETL_ROLE;
```

```
CREATE ROLE ANALYST_ROLE;
```

#### **Explanation:**

Roles represent job functions, not individuals.

---



### Step 3: Attach Custom Roles to SYSADMIN

```
GRANT ROLE ETL_ROLE TO ROLE SYSADMIN;
```

```
GRANT ROLE ANALYST_ROLE TO ROLE SYSADMIN;
```

#### Explanation:

This ensures SYSADMIN and ACCOUNTADMIN inherit all privileges.

---

### Step 4: Create Database & Schema (SYSADMIN)

```
CREATE DATABASE SALES_DB;
```

```
CREATE SCHEMA SALES_DB.RAW;
```

```
CREATE SCHEMA SALES_DB.CURATED;
```

#### Explanation:

- RAW → ETL writes data
  - CURATED → Analytics reads data
- 

### Step 5: Grant Database & Schema Usage

```
GRANT USAGE ON DATABASE SALES_DB TO ROLE ETL_ROLE;
```

```
GRANT USAGE ON DATABASE SALES_DB TO ROLE ANALYST_ROLE;
```

```
GRANT USAGE ON SCHEMA SALES_DB.RAW TO ROLE ETL_ROLE;
```

```
GRANT USAGE ON SCHEMA SALES_DB.CURATED TO ROLE ETL_ROLE;
```

```
GRANT USAGE ON SCHEMA SALES_DB.CURATED TO ROLE ANALYST_ROLE;
```

#### Explanation:

Without USAGE, roles cannot see objects.

---

### Step 6: Grant Table-Level Privileges

#### ETL Role (Full Control)

```
GRANT SELECT, INSERT, UPDATE, DELETE
```

```
ON ALL TABLES IN SCHEMA SALES_DB.RAW
```

```
TO ROLE ETL_ROLE;
```

#### Analyst Role (Read Only)

```
GRANT SELECT
```

```
ON ALL TABLES IN SCHEMA SALES_DB.CURATED
```

```
TO ROLE ANALYST_ROLE;
```

### Step 7: Future Grants (Critical for Real Projects)

```
GRANT SELECT, INSERT, UPDATE, DELETE  
ON FUTURE TABLES IN SCHEMA SALES_DB.RAW  
TO ROLE ETL_ROLE;
```

```
GRANT SELECT  
ON FUTURE TABLES IN SCHEMA SALES_DB.CURATED  
TO ROLE ANALYST_ROLE;
```

#### Explanation:

Ensures access continues when new tables are created.

---

### Step 8: Create Users (USERADMIN)

```
CREATE USER etl_user  
PASSWORD='Temp@123'  
DEFAULT_ROLE=ETL_ROLE  
MUST_CHANGE_PASSWORD=TRUE;
```

```
CREATE USER analyst_user  
PASSWORD='Temp@123'  
DEFAULT_ROLE=ANALYST_ROLE  
MUST_CHANGE_PASSWORD=TRUE;
```

---

### Step 9: Assign Roles to Users

```
GRANT ROLE ETL_ROLE TO USER etl_user;  
GRANT ROLE ANALYST_ROLE TO USER analyst_user;
```

#### Explanation:

Users inherit privileges only through roles.

---

## Step 10: Validate Access

### ETL User

```
USE ROLE ETL_ROLE;
```

```
INSERT INTO SALES_DB.RAW.SALES VALUES (...);
```

✓ Allowed

### Analyst User

```
USE ROLE ANALYST_ROLE;
```

```
SELECT * FROM SALES_DB.CURATED.SALES_SUMMARY;
```

✓ Allowed

```
INSERT INTO SALES_DB.CURATED.SALES_SUMMARY VALUES (...);
```

✗ Not allowed

---

## Step 11: Revoke Access (If Needed)

```
REVOKE SELECT
```

```
ON ALL TABLES IN SCHEMA SALES_DB.CURATED
```

```
FROM ROLE ANALYST_ROLE;
```

### Explanation:

Immediately removes access for all analysts.

---

## Step 12: How This Works in Real Companies

- ETL tools use **service accounts** with ETL\_ROLE
  - Analysts use SSO + MFA with ANALYST\_ROLE
  - DBAs rarely use ACCOUNTADMIN
  - Access changes are ticket-driven
- 

## Common Interview Questions from This Scenario

1. Why not grant access directly to users?
2. Why are future grants required?
3. Why attach custom roles to SYSADMIN?
4. Difference between RAW and CURATED schemas?
5. What happens if future grants are missed?

## Key Takeaways

- Roles represent teams, not people
- Future grants are mandatory
- SYSADMIN is the owner of data
- ACCOUNTADMIN is for governance
- This design is **production-safe & interview-safe**

---

This is a **complete real-time Snowflake RBAC implementation**.

---

## Why this scenario is important

This is **not theory** — this is how RBAC is implemented in:

- Production Snowflake accounts
- Banking / Retail / Analytics projects
- Snowflake DBA & Data Engineer interviews

## What the scenario covers

- ✓ Role design (ETL vs Analyst)
- ✓ Proper use of SYSADMIN, SECURITYADMIN, USERADMIN
- ✓ Database & schema separation (RAW vs CURATED)
- ✓ Step-by-step SQL flow
- ✓ Existing + **future grants**
- ✓ User creation & role assignment
- ✓ Validation (what works / what fails)
- ✓ Real company practices
- ✓ Interview questions derived from the scenario

## How you should study this

1. **Read once end-to-end** (understand flow)
2. **Practice typing SQL** in Snowflake
3. Memorize:
  - Why future grants are required
  - Why roles go under SYSADMIN
  - Why users never get direct grants